Bitswap 프로토콜 동작 원리

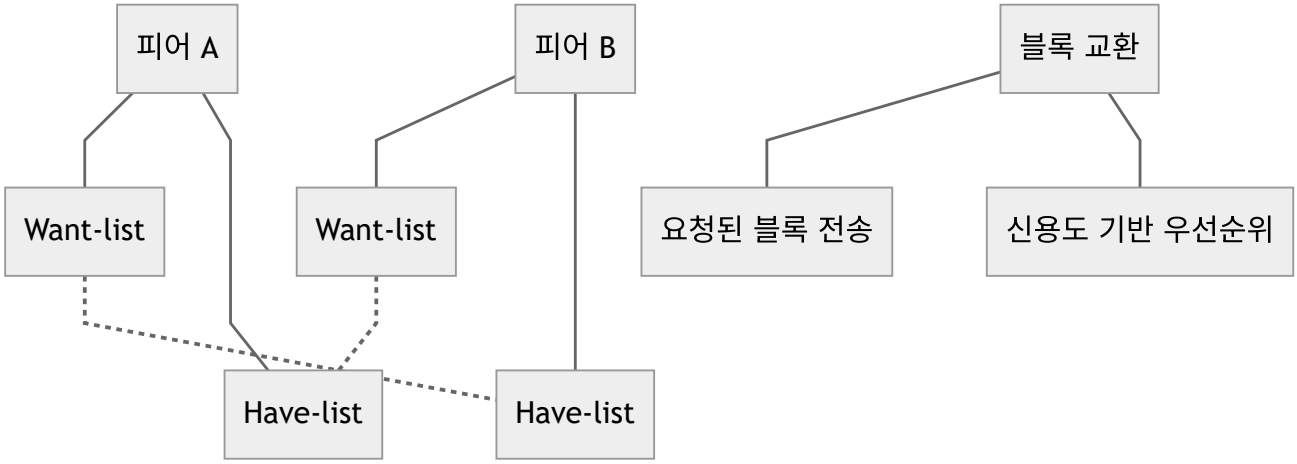
개요

Bitswap은 IPFS의 핵심 데이터 교환 프로토콜로, 피어 간 블록 단위의 데이터 교환을 담당합니다. 이 문서는 Bitswap 프로토콜의 동작 원리, Want-list 관리, 블록 교환 전략, 그리고 성능 최적화 방법을 상세히 설명합니다.

Bitswap 프로토콜 기본 구조

프로토콜 개념

Bitswap은 “tit-for-tat” 전략을 기반으로 한 블록 교환 프로토콜입니다:



핵심 구성 요소

1. **Want-list**: 노드가 원하는 블록들의 목록
2. **Have-list**: 노드가 가지고 있는 블록들의 목록
3. **Ledger**: 피어 간 교환 기록 및 신용도

4. **Strategy**: 블록 전송 우선순위 결정 알고리즘

Bitswap 초기화 및 설정

기본 설정

```
const (
DefaultEngineBlockstoreWorkerCount = 128
DefaultTaskWorkerCount = 8
DefaultEngineTaskWorkerCount = 8
DefaultMaxOutstandingBytesPerPeer = 1 << 20  // 1MB
DefaultProviderSearchDelay = 1000 * time.Millisecond
DefaultMaxProviders = 10
DefaultWantHaveReplaceSize = 1024
)
```

Bitswap 인스턴스 생성

IPFS에서 Bitswap은 다음과 같이 초기화됩니다:

```
func  Bitswap(serverEnabled, libp2pEnabled, httpEnabled  bool) interface{} {
return  func(in bitswapIn, lc fx.Lifecycle) (*bitswap.Bitswap, error) {
var  bitswapNetworks network.BitSwapNetwork
var  bitswapBlockstore blockstore.Blockstore = in.Bs
```

```
connEvtMgr := network.NewConnectEventManager()


// libp2p 네트워크 설정
if libp2pEnabled {
bitswapLibp2p = bsnet.NewFromIpfsHost(in.Host,
bsnet.WithConnectEventManager(connEvtMgr),
)
}


// HTTP 네트워크 설정
if httpEnabled {
bitswapHTTP := httpnet.New(in.Host,
httpnet.WithHTTPWorkers(numWorkers),
httpnet.WithAllowlist(allowlist),
httpnet.WithDenylist(denylist),
httpnet.WithMaxBlockSize(maxBlockSize),
)
bitswapNetworks = network.New(in.Host.Peerstore(), bitswapLibp2p, bitswapHTTP)
}


// 서버 모드 설정
opts = append(opts, bitswap.WithServerEnabled(serverEnabled))
```

```
// Bitswap 인스턴스 생성

bs := bitswap.New(ctx, bitswapNetworks, providerQueryMgr, bitswapBlockstore, opt

return  bs, nil

}

}
```

설정 옵션

기본 Bitswap 설정

```
type Bitswap struct {

Libp2pEnabled Flag // libp2p를 통한 Bitswap 활성화 (기본: true)

ServerEnabled Flag // 다른 피어의 요청에 응답 (기본: true)

}
```

고급 내부 설정

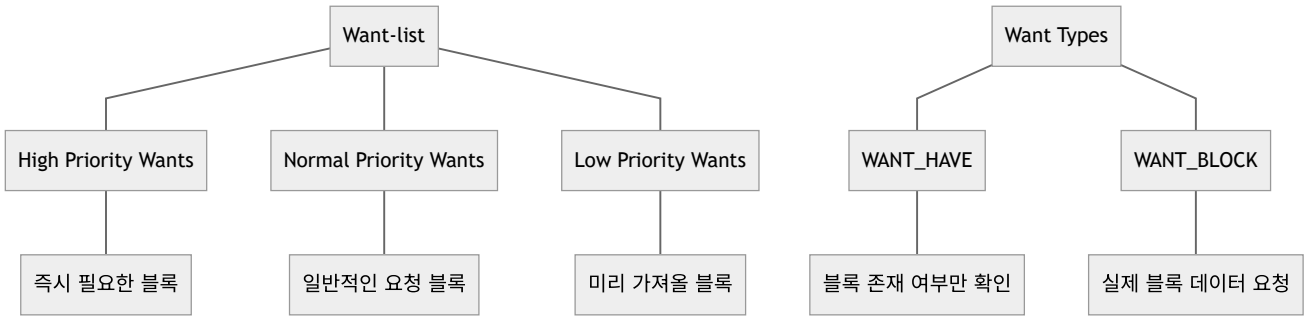
```
type InternalBitswap struct {

TaskWorkerCount OptionalInteger // 태스크 워커 수

EngineBlockstoreWorkerCount OptionalInteger // 블록스토어 워커 수

EngineTaskWorkerCount OptionalInteger // 엔진 태스크 워커 수

MaxOutstandingBytesPerPeer OptionalInteger // 피어당 최대 미처리 바이트

ProviderSearchDelay OptionalDuration // 제공자 검색 지연

ProviderSearchMaxResults OptionalInteger // 최대 제공자 수

WantHaveReplaceSize OptionalInteger // Want-Have 대체 크기
```

```
    BroadcastControl *BitswapBroadcastControl // 브로드캐스트 제어

}
```

Want-list 관리 시스템

Want-list 구조

Want-list는 노드가 필요로 하는 블록들의 우선순위 큐입니다:



Want-list 업데이트 과정

1. **블록 요청**: 애플리케이션이 특정 CID의 블록 요청
2. **Want-list 추가**: 해당 CID를 Want-list에 추가
3. **피어 브로드캐스트**: 연결된 피어들에게 Want-list 전송
4. **응답 처리**: 피어들로부터 블록 또는 HAVE 메시지 수신
5. **Want-list 정리**: 받은 블록을 Want-list에서 제거

Want-Have vs Want-Block

Want-Have 메시지

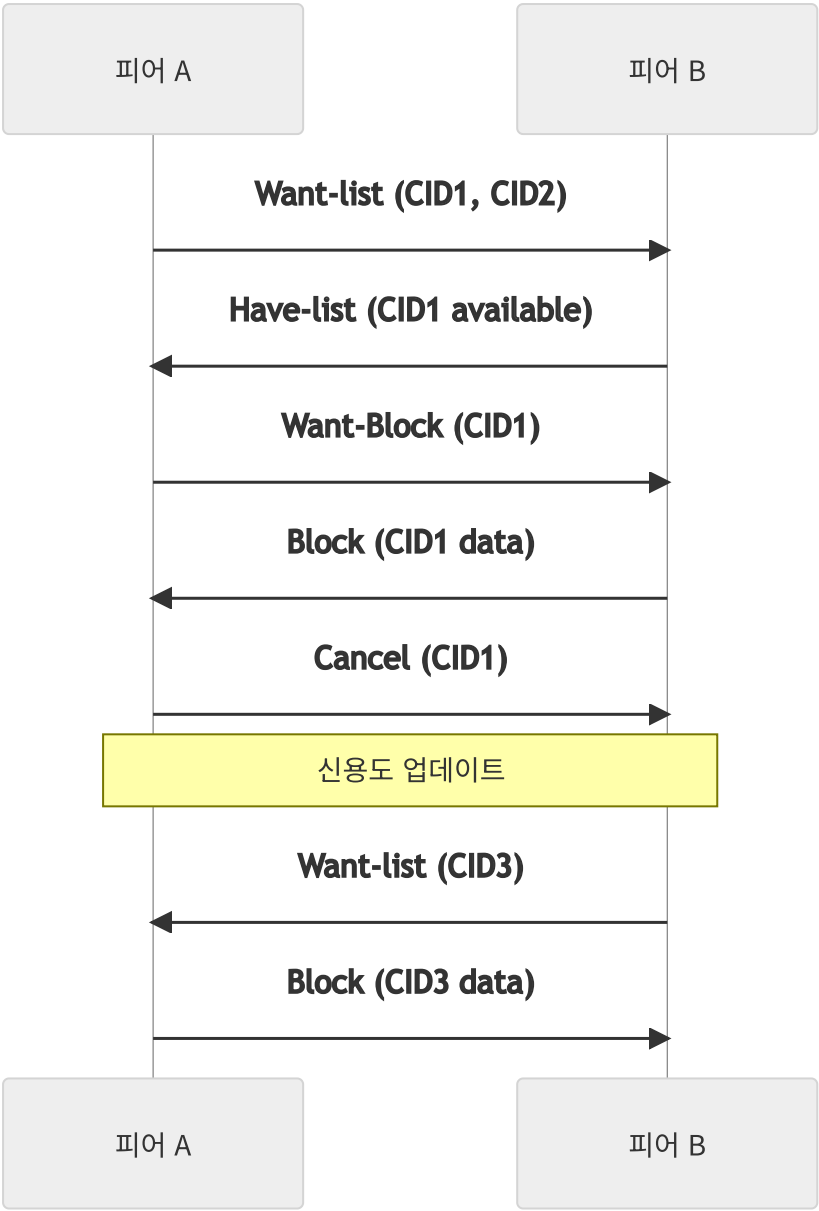
- **목적**: 블록의 존재 여부만 확인
- **장점**: 네트워크 대역폭 절약
- **동작**: 블록 크기가 `WantHaveReplaceSize` (기본 1024바이트)보다 작으면 자동으로 블록 데이터 포함

Want-Block 메시지

- **목적**: 실제 블록 데이터 요청
- **사용**: 즉시 필요한 블록에 사용
- **우선순위**: 높은 우선순위로 처리

블록 교환 메커니즘

교환 과정



블록 전송 우선순위

Bitswap은 다음 요소들을 고려하여 블록 전송 우선순위를 결정합니다:

1. **피어 신용도**: 과거 교환 기록 기반
2. **요청 우선순위**: Want 메시지의 priority 필드
3. **블록 크기**: 작은 블록 우선 전송
4. **네트워크 상태**: 대역폭 및 지연시간 고려

신용도 시스템 (Ledger)

각 피어와의 교환 기록을 추적하여 신용도를 계산합니다:

```
type Ledger struct {
BytesSent  uint64  // 전송한 바이트 수
BytesReceived  uint64  // 수신한 바이트 수
Timestamp time.Time // 마지막 교환 시간

// 계산된 값들
DebtRatio  float64  // 부채 비율
Score  float64  // 전체적인 신용도 점수
}
```

신용도 계산 알고리즘

1. **부채 비율 계산**: `DebtRatio = BytesSent / (BytesReceived + 1)`
2. **시간 가중치**: 최근 교환에 더 높은 가중치 부여
3. **점수 정규화**: 0-1 범위로 정규화하여 비교 가능한 점수 생성

브로드캐스트 제어 시스템

브로드캐스트 제어 개념

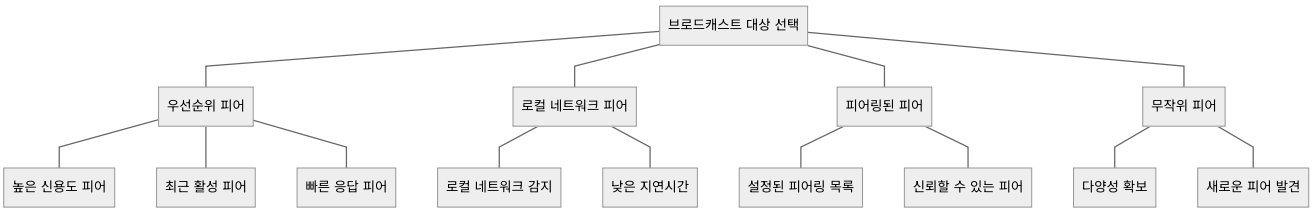
브로드캐스트 제어는 Want-list를 모든 피어에게 보내는 대신, 효율적인 피어 선택을 통해 네트워크 부하를 줄이는 메커니즘입니다.

브로드캐스트 제어 설정

```
type BitswapBroadcastControl struct {
```

```
Enable Flag // 브로드캐스트 제어 활성화 (기본: true)

MaxPeers OptionalInteger // 최대 브로드캐스트 피어 수 (기본: -1, 무제한)

LocalPeers Flag // 로컬 피어 제어 여부 (기본: false)

PeeredPeers Flag // 피어링된 피어 제어 여부 (기본: false)

MaxRandomPeers OptionalInteger // 무작위 피어 수 (기본: 0)

SendToPendingPeers Flag // 대기 중인 피어에게 전송 (기본: false)

}
```

피어 선택 전략



브로드캐스트 피어 선택 알고리즘

1. **피어링된 피어**: 설정에서 지정된 신뢰할 수 있는 피어들
2. **로컬 피어**: 같은 네트워크의 피어들 (지연시간 최소화)
3. **고성능 피어**: 높은 신용도와 빠른 응답 시간을 가진 피어들
4. **무작위 피어**: 네트워크 다양성 확보를 위한 무작위 선택

브로드캐스트 최적화 구현

```
func BitswapOptions(cfg *config.Config) interface{} {
return func() bitswapOptionsOut {
var internalBsCfg config.InternalBitswap
if cfg.Internal.Bitswap != nil {
internalBsCfg = *cfg.Internal.Bitswap
}

// 브로드캐스트 제어 설정
if bcCfg := internalBsCfg.BroadcastControl; bcCfg != nil {
bcEnable := bcCfg.Enable.WithDefault(config.DefaultBroadcastControlEnable)
opts = append(opts, bitswap.WithClientOption(client.BroadcastControlEnable(bcEna

if bcEnable {
// 최대 피어 수 설정
maxPeers := int(bcCfg.MaxPeers.WithDefault(config.DefaultBroadcastControlMaxPeer
opts = append(opts, bitswap.WithClientOption(client.BroadcastControlMaxPeers(max

// 로컬 피어 제어 설정
localPeers := bcCfg.LocalPeers.WithDefault(config.DefaultBroadcastControlLocalPe
opts = append(opts, bitswap.WithClientOption(client.BroadcastControlLocalPeers(l

// 무작위 피어 수 설정
```

```
maxRandomPeers := int(bcCfg.MaxRandomPeers.WithDefault(config.DefaultBroadcastCo

opts = append(opts, bitswap.WithClientOption(client.BroadcastControlMaxRandomPee

}

}



return bitswapOptionsOut{BitswapOpts: opts}

}

}
```

브로드캐스트 최적화 효과

- **네트워크 부하 감소**: 불필요한 메시지 전송 최소화
- **응답 시간 개선**: 효율적인 피어에게 우선 요청
- **대역폭 절약**: 중복 요청 및 응답 감소
- **확장성 향상**: 대규모 네트워크에서의 성능 유지

HTTP 기반 블록 검색

HTTP 검색 통합

IPFS는 libp2p 기반 Bitswap과 함께 HTTP 기반 블록 검색을 지원합니다:

```
if  httpEnabled {

httpCfg := in.Cfg.HTTPRetrieval

maxBlockSize, err := humanize.ParseBytes(httpCfg.MaxBlockSize.WithDefault(config
```

```
bitswapHTTP := httpnet.New(in.Host,
httpnet.WithHTTPWorkers(int(httpCfg.NumWorkers.WithDefault(config.DefaultHTTPRet
httpnet.WithAllowlist(httpCfg.Allowlist),
httpnet.WithDenylist(httpCfg.Denylist),
httpnet.WithInsecureSkipVerify(httpCfg.TLSInsecureSkipVerify.WithDefault(config.
httpnet.WithMaxBlockSize(int64(maxBlockSize)),
httpnet.WithUserAgent(version.GetUserAgentVersion()),
)
bitswapNetworks = network.New(in.Host.Peerstore(), bitswapLibp2p, bitswapHTTP)
}
```

HTTP 검색 설정

```
type HTTPRetrieval struct {
Enabled Flag // HTTP 검색 활성화
NumWorkers OptionalInteger // HTTP 워커 수 (기본: 16)
MaxBlockSize OptionalString // 최대 블록 크기 (기본: "2MiB")
TLSInsecureSkipVerify Flag // TLS 검증 건너뛰기 (테스트용)
Allowlist []string  // 허용된 도메인 목록
Denylist []string  // 차단된 도메인 목록
}
```

HTTP 검색 특징

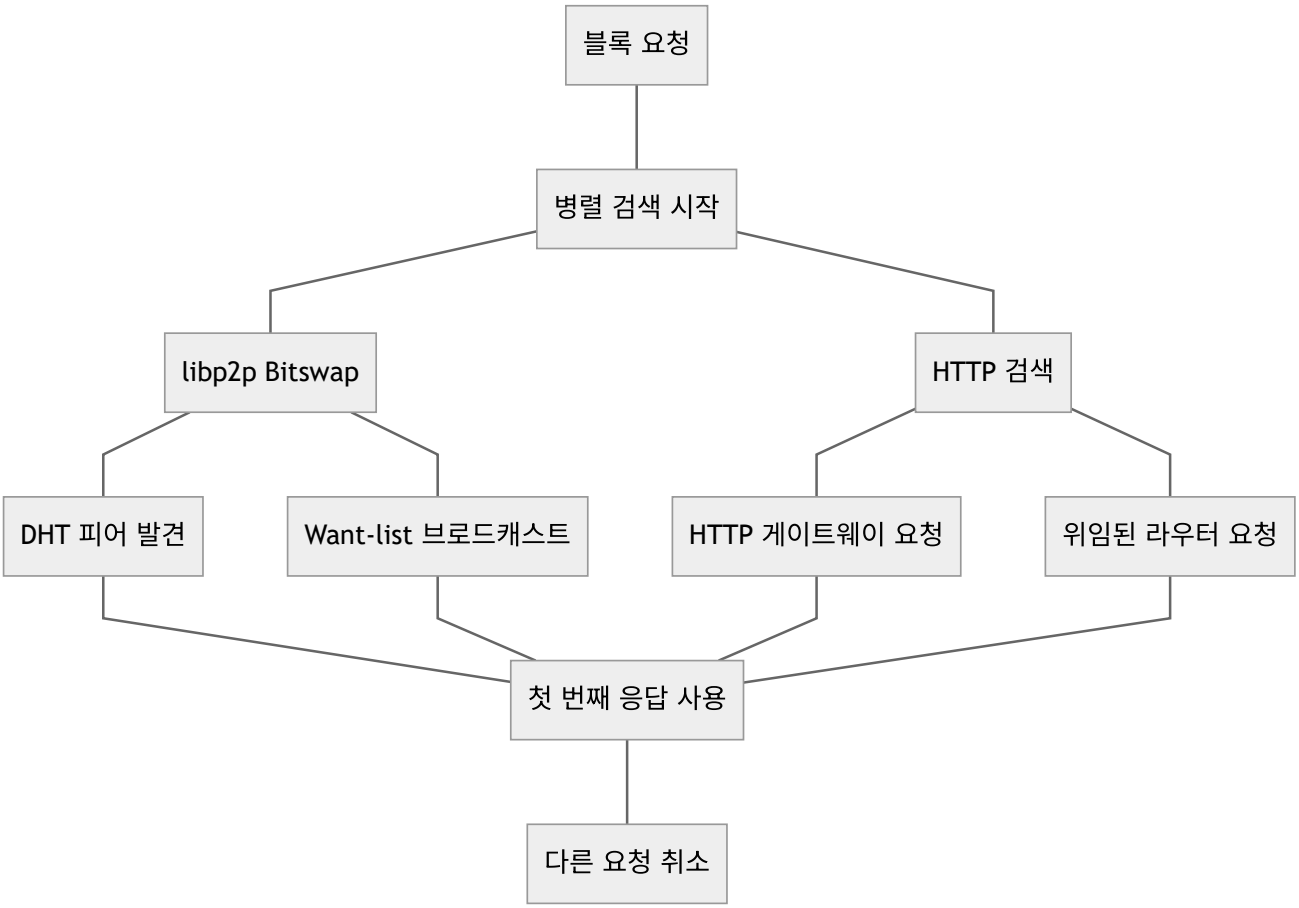
장점

- **빠른 응답**: 중앙화된 서버의 빠른 응답 시간
- **높은 가용성**: 전문적으로 관리되는 서버 인프라
- **웹 호환성**: 브라우저 기반 노드와의 호환성

단점

- **중앙화 의존성**: 특정 서버에 대한 의존성
- **프라이버시 우려**: 요청 기록이 서버에 남을 수 있음
- **검열 가능성**: 서버 운영자에 의한 콘텐츠 차단 가능

하이브리드 검색 전략



제공자 쿼리 관리

제공자 쿼리 매니저

IPFS는 커스터마이징된 제공자 쿼리 매니저를 사용합니다:

```
// Kubo는 자체 제공자 쿼리 매니저 사용
opts = append(opts, bitswap.WithClientOption(client.WithDefaultProviderQueryMana
```

```
// 무시할 제공자 설정
ignoredPeerIDs := make([]peer.ID, 0, len(cfg.Routing.IgnoreProviders))
```

```
for _, str := range cfg.Routing.IgnoreProviders {
pid, err := peer.Decode(str)
if err != nil {
return nil, err
}
ignoredPeerIDs = append(ignoredPeerIDs, pid)
}

// 제공자 쿼리 매니저 생성
providerQueryMgr, err := rpqm.New(bitswapNetworks, discovery,
rpqm.WithMaxProviders(maxProviders),
rpqm.WithIgnoreProviders(ignoredPeerIDs...),
)
```

제공자 검색 최적화

- **검색 지연**: `ProviderSearchDelay` 로 검색 빈도 조절
- **최대 제공자 수**: `MaxProviders` 로 검색 범위 제한
- **제공자 필터링**: 신뢰할 수 없는 제공자 차단

성능 최적화 전략

워커 스레드 최적화

```
// 블록스토어 워커: 블록 읽기/쓰기 처리
EngineBlockstoreWorkerCount: 128

// 태스크 워커: 일반적인 비동기 작업 처리
TaskWorkerCount: 8

// 엔진 태스크 워커: Bitswap 엔진 내부 작업 처리
EngineTaskWorkerCount: 8
```

워커 수 조정 가이드

- **CPU 집약적 환경**: 워커 수를 CPU 코어 수의 2-4배로 설정
- **I/O 집약적 환경**: 더 많은 워커로 I/O 대기 시간 활용
- **메모리 제약 환경**: 워커 수를 줄여 메모리 사용량 제한

대역폭 관리

```
// 피어당 최대 미처리 바이트 수
MaxOutstandingBytesPerPeer: 1MB

// 제공자 검색 지연 시간
ProviderSearchDelay: 1000ms

// 최대 제공자 수
```

```
MaxProviders: 10
```

대역폭 최적화 전략

1. **적응적 윈도우 크기**: 네트워크 상황에 따른 동적 조정
2. **우선순위 기반 전송**: 중요한 블록 우선 전송
3. **배치 처리**: 여러 블록을 묶어서 효율적 전송
4. **압축**: 가능한 경우 블록 데이터 압축

캐싱 전략

블록 캐싱

- **LRU 캐시**: 최근 사용된 블록 우선 유지
- **크기 기반 제한**: 메모리 사용량 제한
- **TTL 관리**: 오래된 블록 자동 정리

피어 정보 캐싱

- **성능 메트릭**: 피어별 응답 시간 및 처리량 기록
- **신뢰도 점수**: 과거 교환 기록 기반 점수 유지
- **가용성 정보**: 피어의 온라인 상태 추적

문제 해결 및 디버깅

일반적인 문제들

1. 느린 블록 검색

원인:

- 네트워크 연결 문제
- 피어 부족
- 제공자 부족

해결 방법:

```
# 연결된 피어 확인
ipfs swarm peers

# DHT 제공자 검색
ipfs dht findprovs <CID>

# Bitswap 통계 확인
ipfs bitswap stat
```

2. 높은 메모리 사용량

원인:

- 과도한 워커 스레드
- 큰 Want-list
- 캐시 크기 문제

해결 방법:

```
{
"Internal": {
"Bitswap": {
```

```
"TaskWorkerCount": 4,
"EngineBlockstoreWorkerCount": 64,
"MaxOutstandingBytesPerPeer": 524288
}
}
}
```


3. 네트워크 과부하

원인:

- 브로드캐스트 제어 비활성화
- 과도한 피어 연결
- 비효율적인 Want-list 관리

해결 방법:


```
{
"Internal": {
"Bitswap": {
"BroadcastControl": {
"Enable": true,
"MaxPeers": 20,
"MaxRandomPeers": 5
}
}
```

```
}

}
```

성능 모니터링

Bitswap 통계 확인

```
# 전체 Bitswap 통계
ipfs bitswap stat

# 특정 피어와의 교환 통계
ipfs bitswap ledger <peer-id>

# Want-list 확인
ipfs bitswap wantlist
```

로그 분석

```
# Bitswap 디버그 로그 활성화
export IPFS_LOGGING=debug
export IPFS_LOGGING_FMT=color

# 특정 컴포넌트 로그만 활성화
ipfs log level bitswap debug
```

보안 고려사항

1. DoS 공격 방어

요청 제한

- **피어당 요청 수 제한**: 과도한 요청 방지
- **블록 크기 제한**: 메모리 고갈 공격 방지 (기본 2MiB)
- **대역폭 제한**: 네트워크 자원 보호

신용도 시스템 활용

- **악의적 피어 식별**: 비정상적인 행동 패턴 감지
- **자동 차단**: 신용도가 낮은 피어 자동 차단
- **복구 메커니즘**: 일시적 문제로 차단된 피어의 복구

2. 프라이버시 보호

요청 패턴 난독화

- **더미 요청**: 실제 요청과 함께 더미 요청 전송
- **지연 요청**: 요청 시간을 무작위로 지연
- **배치 요청**: 여러 요청을 묶어서 패턴 숨김

메타데이터 최소화

- **필요한 정보만 전송**: 불필요한 메타데이터 제거
- **익명화**: 가능한 경우 요청자 정보 익명화

3. 콘텐츠 검증

블록 무결성 검증

```
// CID 기반 검증
func VerifyBlock(block blocks.Block) error {
expectedCID := block.Cid()
actualCID, err := cid.Prefix{
Version: expectedCID.Version(),
Codec: expectedCID.Type(),
MhType: expectedCID.Hash().Code,
MhLength: expectedCID.Hash().Length,
}.Sum(block.RawData())

if !expectedCID.Equals(actualCID) {
return fmt.Errorf("block verification failed")
}
return nil
}
```

설정 예제

기본 설정

```
{
"Bitswap": {
"Libp2pEnabled": true,
```

```
"ServerEnabled": true
},
"HTTPRetrieval": {
"Enabled": true,
"NumWorkers": 16,
"MaxBlockSize": "2MiB"
}
}
```


고성능 서버 설정


```
{
"Internal": {
"Bitswap": {
"TaskWorkerCount": 16,
"EngineBlockstoreWorkerCount": 256,
"EngineTaskWorkerCount": 16,
"MaxOutstandingBytesPerPeer": 2097152,
"ProviderSearchDelay": "500ms",
"ProviderSearchMaxResults": 20,
"BroadcastControl": {
"Enable": true,
"MaxPeers": 50,
```

```
"MaxRandomPeers": 10
}
}
}
}
```

저사양 클라이언트 설정

```
{
"Bitswap": {
"ServerEnabled": false
},
"Internal": {
"Bitswap": {
"TaskWorkerCount": 2,
"EngineBlockstoreWorkerCount": 32,
"EngineTaskWorkerCount": 2,
"MaxOutstandingBytesPerPeer": 262144,
"BroadcastControl": {
"Enable": true,
"MaxPeers": 10,
"MaxRandomPeers": 2
}
```

```
}

}

}
```

미래 개선 방향

1. 프로토콜 최적화

GraphSync 통합

- **효율적인 DAG 동기화**: 전체 DAG 구조를 한 번에 요청
- **선택적 동기화**: 필요한 부분만 선택적으로 동기화
- **증분 업데이트**: 변경된 부분만 업데이트

압축 및 중복 제거

- **블록 레벨 압축**: 개별 블록 압축으로 대역폭 절약
- **중복 제거**: 동일한 블록의 중복 전송 방지
- **델타 압축**: 유사한 블록 간의 차이만 전송

2. 인센티브 메커니즘

토큰 기반 교환

- **마이크로페이먼트**: 블록 교환에 대한 소액 결제
- **평판 시스템**: 장기적인 평판 기반 우선순위
- **자원 기여도**: 네트워크 기여도에 따른 혜택

경제적 모델

- **대역폭 시장**: 대역폭 자원의 시장 기반 할당
- **저장 인센티브**: 장기 저장에 대한 인센티브 제공
- **품질 보상**: 높은 품질의 서비스 제공에 대한 보상

이 문서는 IPFS Bitswap 프로토콜의 동작 원리와 최적화 방법에 대한 포괄적인 이해를 제공합니다. Bitswap은 IPFS 네트워크의 핵심 구성 요소로서, 효율적인 데이터 교환을 통해 분산 파일 시스템의 성능을 결정하는 중요한 역할을 합니다.